React Interview Cheat Sheet

React Interview Cheat Sheet

1. React Fundamentals

React is a JavaScript library for building user interfaces, maintained by Meta. It is not a full framework — it focuses on the **view layer**.

Key characteristics - Component-based: UI built from reusable, self-contained components - Declarative: you describe *what* the UI should look like; React handles *how* to update the DOM - Unidirectional data flow: props flow parent → child - Cross-platform: React Native (mobile), Next.js (server), etc.

Problem	React Solution
Manual DOM manipulation	Declarative components
State drift across UI	Single source of truth (state)
Code complexity in large apps	Components encapsulate logic + UI
Slow performance with frequent updates	Virtual DOM batching

2. Virtual DOM & Reconciliation

The Virtual DOM (VDOM) is a lightweight JS representation of the real DOM, used to minimize expensive real-DOM writes.

Lifecycle 1. **Render** — state changes → new VDOM tree created (cheap JS objects) 2. **Diffing (Reconciliation)** — new VDOM compared against previous VDOM, O(n) tree walk 3. **Commit** — only the minimal set of real DOM changes is applied, batched

```
// Real DOM (expensive):
document.getElementById("app").innerHTML = "<h1>Hello</h1><p>World</p>";

// Virtual DOM (cheap JS objects):
const vdom = {
  type: "div",
  props: { children: [{ type: "h1", props: { children: ["Hello"] } }] }
};
```

Diffing rules: components of different types tear down the subtree; the `key` prop identifies list items across renders so React can reuse them instead of destroying and recreating.

Notes - Not always faster than direct DOM manipulation for trivial, rarely-updated apps - React 18 introduces **Concurrent Rendering** — diffing can pause/resume to prioritize urgent updates - **React Fiber** (since React 16) is the reconciliation engine enabling incremental, interruptible rendering

Reconciliation, in one line

Reconciliation is the process React uses to compare the previous VDOM with the new VDOM, identify differences, and update only the changed parts of the real DOM. Fiber is the engine that performs this efficiently, in two phases: **Render phase** (calculate changes, interruptible) and **Commit phase** (apply changes to the real DOM, synchronous).

3. Hooks

3.1 useState

```
const [state, setState] = useState(initialValue);
```

- Calling the setter triggers a re-render
- State is preserved across renders, per component instance
- Updates may be **batched**
- Setter accepts a value or an updater function: `setCount(prev => prev + 1)` — always prefer the updater form when new state depends on old state

```
// Objects/arrays need a new reference (shallow merge is manual):
setUser(prev => ({ ...prev, name: "New Name" }));
setItems(prev => [...prev, newItem]);
```

Rules of Hooks: only call at the top level (never in loops/conditions), only call from function components or custom hooks, call order must stay consistent across renders.

Scenario	Use useState?
Primitive value, form input, toggle	Yes
Complex state logic / multiple sub-values	No — use `useReducer`
Cross-component shared state	No — use Context

3.2 useEffect

Pattern	Code
Mount once	`useEffect(fn, [])`
Every render	`useEffect(fn)`
On value change	`useEffect(fn, [name])`
Cleanup	return a function — removes listeners, clears timers

```
useEffect(() => {
  const controller = new AbortController();
  fetch("/api/users", { signal: controller.signal })
    .then(res => res.json())
    .then(setUsers);
  return () => controller.abort(); // cleanup
}, []);
```

3.3 useLayoutEffect vs useEffect *(added)*

Both run side effects after render, but at different times:

	useEffect	useLayoutEffect
Timing	Runs **after** the browser paints	Runs **before** the browser paints, synchronously
Blocks paint?	No	Yes
Use for	Data fetching, subscriptions, logging	Measuring/mutating the DOM before the user sees it (e.g. avoiding layout flicker)

Rule of thumb: default to `useEffect`; reach for `useLayoutEffect` only when you must read layout (like `getBoundingClientRect`) and synchronously adjust the DOM before paint.

3.4 useRef

`useRef` returns a mutable `{ current }` object that persists across renders. Changing `.current` does **not** trigger a re-render.

```
const inputRef = useRef(null);
useEffect(() => { inputRef.current?.focus(); }, []);
return <input ref={inputRef} />;
```

Use cases: DOM access, storing timer/interval IDs, storing a previous value, tracking render counts.

Feature	useState	useRef
Triggers re-render?	Yes	No
Purpose	Data that affects UI	Mutate values / access DOM
Access	Direct	Via `.current`

Pitfalls: expecting a ref change to update the UI (won't — use state instead); forgetting to null-check DOM refs before mount; using a ref for derived/computed values (goes stale — use direct computation or `useMemo`).

3.5 forwardRef & useImperativeHandle *(added)*

Refs don't pass through components by default — `forwardRef` lets a parent get a ref to a child's DOM node or an imperative API.

```
const FancyInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(),
    clear: () => (inputRef.current.value = "")
  }));
  return <input ref={inputRef} {...props} />;
});

// Parent
const ref = useRef(null);
<FancyInput ref={ref} />
ref.current.focus(); // calls the exposed method, not the raw DOM node
```

Use sparingly — only when a parent genuinely needs to imperatively command a child (focus, scroll, animation triggers), not as a substitute for props/state.

3.6 useContext

Lets components read a value from Context without prop drilling.

3-step pattern

```
// 1. Create (module level, not inside a component)
const ThemeContext = createContext("light");

// 2. Provide
function App() {
  const [theme, setTheme] = useState("light");
  return (
    <ThemeContext.Provider value={{ theme, toggleTheme: () => setTheme(p => p === "light" ? "dark" : "light") }}>
      <Header />
    </ThemeContext.Provider>
  );
}

// 3. Consume
function Header() {
  const { theme, toggleTheme } = useContext(ThemeContext);
  return <header style={{ background: theme }}>...</header>;
}
```

Inner providers override outer providers for the same context. Split contexts by concern (theme, auth, i18n) to avoid re-rendering every consumer when only one field changes; stabilize object values with `useMemo`.

Scenario	Use Context?
Theme / auth / i18n / feature flags	Yes
Data passed 2–3 levels	No — just pass props
Frequently changing data (scroll position)	No — causes wide re-renders
Complex global state	Consider Zustand/Redux Toolkit

Common pitfalls: creating context inside a component (breaks identity every render); passing a whole object as `value` without `useMemo` (re-renders all consumers on any field change); missing a default value (SSR/hydration mismatch).

3.7 useReducer

Alternative to `useState` for complex logic, multiple sub-values, or nested updates. Follows the Redux pattern: a pure `(state, action) => newState` function.

```
function reducer(state, action) {
  switch (action.type) {
    case "ADD_TODO":
      return { ...state, todos: [...state.todos, action.payload] };
    case "TOGGLE_TODO":
      return { ...state, todos: state.todos.map(t => t.id === action.payload ? { ...t, completed: !t.completed } : t) };
    default:
      return state; // safety net for unknown actions
  }
}
const [state, dispatch] = useReducer(reducer, { todos: [], filter: "all" });
dispatch({ type: "ADD_TODO", payload: "Buy milk" });
```

`dispatch` is guaranteed stable across renders — safe to omit from dependency arrays.

Lazy initialization — pass a function as the second arg to defer expensive initial state (e.g. reading `localStorage`):

```
const [state, dispatch] = useReducer(reducer, () => JSON.parse(localStorage.getItem("todos")) ?? initialState);
```

Feature	useState	useReducer
Best for	Simple, independent values	Complex/related state, multiple sub-values
Update syntax	`setState(value)`	`dispatch({ type, payload })`
Testing	Handler + state together	Reducer alone, pure function
Pairs with	—	`useContext` for global state

3.8 useMemo

Memoizes a **computed value**; only recalculates when dependencies change.

```
const filteredTodos = useMemo(() => {
  return todos
    .filter(t => filter === "all" || t.completed === (filter === "completed"))
    .filter(t => t.text.toLowerCase().includes(searchTerm.toLowerCase()));
}, [todos, filter, searchTerm]);
```

Use for expensive computations (sorting/filtering large lists) or to keep an object/array reference stable for a memoized child. Don't wrap trivial math (`price * quantity`) — the memoization overhead can exceed the savings.

3.9 useCallback *(added — was referenced but never explained)*

Memoizes a **function reference** (not its return value) so it doesn't get recreated on every render — critical when passing callbacks to `React.memo`-wrapped children or as `useEffect` dependencies.

```
const handleClick = useCallback(() => {
  console.log("Clicked!", count);
}, [count]); // new function only when count changes
```

	useMemo	useCallback
Memoizes	A value	A function
Equivalent to	`useMemo(() => fn, deps)`	—
Typical use	Derived data, expensive calc	Stable callback prop for a memoized child

```
// useCallback(fn, deps) is sugar for:
useMemo(() => fn, deps);
```

Pitfall: `useCallback` alone doesn't prevent a child re-render unless the child is wrapped in `React.memo` — the two are almost always used together.

3.10 React.memo *(added — was referenced but never explained)*

Higher-order component that skips re-rendering a component if its props are shallow-equal to the previous render.

```
const Child = React.memo(function Child({ config }) {
  return <div>{config.theme}</div>;
});
// Only re-renders if `config` reference changes — combine with useMemo/useCallback
// on the parent side so config isn't a new object/function every render.
```

Pass a custom comparison function as the second argument for deep-equality needs: `React.memo(Component, (prevProps, nextProps) => boolean)`.

3.11 React 18 concurrent hooks *(added)*

Hook	Purpose
`useTransition`	Marks a state update as low-priority (“transition”), so urgent updates (typing, clicks) aren’t blocked. Returns `[isPending, startTransition]`.
`useDeferredValue`	Defers re-rendering a non-urgent part of the UI until more urgent updates finish — useful for expensive lists driven by a fast-changing input.
`useId`	Generates stable, unique IDs for accessibility attributes (`aria-*`, `htmlFor`) that match between server and client render (SSR-safe).

```
const [isPending, startTransition] = useTransition();
const handleChange = (e) => {
  setInput(e.target.value);           // urgent — keeps input responsive
  startTransition(() => {
    setSearchResults(filterBigList(e.target.value)); // low-priority
  });
};
```

4. Component Patterns *(added)*

4.1 Higher-Order Components (HOC)

A function that takes a component and returns a new, enhanced component. Predates hooks but still appears in libraries (`connect()` from `react-redux`, `withRouter`).

```
function withLoading(Component) {
  return function WithLoading({ isLoading, ...props }) {
    if (isLoading) return <Spinner />;
    return <Component {...props} />;
  };
}
const UserListWithLoading = withLoading(UserList);
```

4.2 Render Props

A component takes a function as a prop (often `children`) and calls it with internal state, letting the caller control rendering.

```
function MouseTracker({ children }) {
  const [pos, setPos] = useState({ x: 0, y: 0 });
  return (
    <div onMouseMove={e => setPos({ x: e.clientX, y: e.clientY })}>
      {children(pos)}
    </div>
  );
}
<MouseTracker>{({ x, y }) => <p>{x}, {y}</p>}</MouseTracker>
```

Custom hooks have largely replaced both patterns for logic reuse — prefer a hook unless you specifically need to control *what gets rendered*, not just *what logic runs*.

4.3 Error Boundaries *(added)*

A class component (no hook equivalent exists yet) that catches JS errors in its child tree, logs them, and renders a fallback UI instead of crashing the whole app.

```
class ErrorBoundary extends React.Component {
  state = { hasError: false };
  static getDerivedStateFromError() { return { hasError: true }; }
  componentDidCatch(error, info) { logErrorToService(error, info); }
  render() {
    if (this.state.hasError) return <h2>Something went wrong.</h2>;
    return this.props.children;
  }
}
<ErrorBoundary><Widget /></ErrorBoundary>
```

Error boundaries do **not** catch errors in event handlers, async code, SSR, or errors thrown in the boundary itself — those need regular `try/catch`.

4.4 Portals *(added)*

Renders children into a DOM node outside the parent hierarchy — used for modals, tooltips, and dropdowns that need to escape a parent's `overflow: hidden` or `z-index` stacking context.

```
import { createPortal } from "react-dom";

function Modal({ children }) {
  return createPortal(
    <div className="modal">{children}</div>,
    document.getElementById("modal-root")
  );
}
```

Events still bubble through the React tree (not the DOM tree), so a portal's `onClick` can still be caught by a React parent listener.

5. Re-renders & Performance

Unnecessary re-renders happen when a component renders even though its displayed output hasn't changed. Common causes: parent re-renders, unneeded state updates, new object/array/function references passed as props each render, context updates, Redux state changes.

Toolbox

Problem	Solution
Child re-renders when props haven't changed	`React.memo`
Expensive calculation repeated every render	`useMemo`
Function prop recreated every render	`useCallback`
Large list rendered at once	`react-window` / virtualization
Too much data fetched	Pagination / infinite scroll
Too many API calls	Debounce / cache
Images loading slowly	Lazy loading
Unstable reconciliation	Stable `key` (use an id, not the array index)

Debugging performance: React DevTools + Profiler to spot components re-rendering too often; Chrome Network tab for slow API calls; Chrome Performance tab for paint/layout cost. Apply `React.memo` / `useMemo` / `useCallback` only where profiling shows a measurable win — premature memoization adds its own overhead.

Why array index as key is risky

```
items.map((item, index) => <li key={index}>{item.name}</li>)
```

If items are added, removed, or reordered, indices shift and React misattributes state/DOM nodes to the wrong item. Use a stable, unique id instead.

6. Rendering Strategies: CSR vs SSR

```
SSR:  Request → Server renders components → HTML sent → Browser paints → React hydrates → Interactive
CSR:  Request → Download HTML shell → Download JS → React renders in browser → Interactive
```

	CSR	SSR
Where rendering happens	Browser	Server
Initial load	Slower (blank until JS loads)	Faster (content visible immediately)
SEO	Weaker	Stronger
Typical stack	Plain React (CRA/Vite)	Next.js

SSR sends pre-rendered HTML so users see content immediately and search engines can crawl it; React then **hydrates** the markup client-side, attaching event listeners to make it interactive.

Sample honest answer if you haven't used it: "I haven't shipped SSR in production, but I understand the concept, its SEO/performance benefits, and that Next.js provides it out of the box."

7. State Management Beyond Component State

7.1 Context + useReducer (no external library)

Good for auth state, theme, language, app-wide settings in small-to-medium apps.

7.2 Redux / Redux Toolkit

Side effects (API calls, timers) live in middleware, not reducers, since reducers must stay pure. `createAsyncThunk` handles the pending/fulfilled/rejected lifecycle automatically.

```
dispatch(fetchUsers()) → createAsyncThunk → API call → pending → fulfilled/rejected → reducer updates store → UI re-renders
```

Redux	Redux Toolkit
More boilerplate	Minimal boilerplate
Manual store setup	`configureStore()`
Manual action types/creators	`createSlice()` generates them
Manual middleware config	Built-in
Async needs extra middleware	`createAsyncThunk()`

7.3 Comparison of options *(added)*

Library	Boilerplate	Learning curve	Best for
Context + useReducer	Low	Low	Small/medium apps, a few global values
Redux Toolkit	Medium	Medium	Large apps, complex async flows, devtools/time-travel debugging
Zustand	Low	Low	Medium apps wanting simple global state without providers
Recoil / Jotai	Low–Medium	Low–Medium	Atom-based state, fine-grained re-renders

---

8. Forms

	Controlled	Uncontrolled
Value stored	React state	The DOM
Access via	`useState`	`useRef`
Who owns the value	React	Browser
Validation	Easy	Harder
Re-renders on change	Yes	No
Typical use	Most forms	Simple forms, file inputs

```
// Controlled
const [value, setValue] = useState("");
<input value={value} onChange={e => setValue(e.target.value)} />;

// Uncontrolled (file inputs must be uncontrolled — browsers won't let React set them)
<input type="file" ref={fileRef} />
```

---

9. Routing *(added)*

React Router is the de facto standard for client-side routing.

```jsx
import { BrowserRouter, Routes, Route, useNavigate, useParams, Outlet } from "react-router-dom";

<BrowserRouter>
  <Routes>
    <Route path="/" element={<Layout />}>
      <Route index element={<Home />} />
      <Route path="users/:id" element={<UserProfile />} />
      <Route path="dashboard" element={
        <ProtectedRoute><Dashboard /></ProtectedRoute>
      } />
    </Route>
  </Routes>
</BrowserRouter>

function UserProfile() {
  const { id } = useParams();       // read the :id URL param
  const navigate = useNavigate();   // navigate programmatically
  return <button onClick={() => navigate(-1)}>Back</button>;
}

function Layout() {
  return <><Header /><Outlet /></>; // Outlet renders the matched child route
}
```

Protected routes — a wrapper that redirects unauthenticated users:

```jsx
function ProtectedRoute({ children }) {
  const { user } = useContext(AuthContext);
  return user ? children : <Navigate to="/login" />;
}
```

Breadcrumbs: read the current path with `useLocation()`, split it into segments, and dynamically generate breadcrumb links back up the hierarchy — improves navigability in deep apps.

---

10. Code Splitting, Lazy Loading & Suspense

Load components only when needed, instead of bundling everything upfront.

```jsx
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./Dashboard"));

<Suspense fallback={<Loader />}>
  <Dashboard />
</Suspense>
```

Common with routes — `Dashboard`'s code downloads only when the user navigates to `/dashboard`, shrinking the initial bundle and speeding up first load. `Suspense` shows the fallback while the lazy chunk downloads.

---

11. Authentication

Typical token-based flow:

```
Login form → POST /login → backend validates → JWT issued
  → token stored (localStorage/httpOnly cookie)
  → Authorization: Bearer <token> on subsequent requests
  → backend validates token → protected data returned
  → logout clears token, redirects to /login
```

Protected routes gate pages behind an auth check (see §9). In production, **HTTP-only cookies** are generally preferred over `localStorage` since they aren't accessible to JS, reducing XSS token-theft risk.

---

12. Security *(added)*

- **XSS via** `dangerouslySetInnerHTML`: React escapes text content by default, but this prop bypasses that — only use it with sanitized HTML (e.g. via `DOMPurify`), never with raw user input.

```
<div dangerouslySetInnerHTML={{ __html: DOMPurify.sanitize(userContent) }} />
```

- Avoid storing sensitive tokens in `localStorage` when possible (readable by any injected script); prefer HTTP-only, `Secure`, `SameSite` cookies.
- Validate/escape data on the server too — client-side checks are for UX, not security.
- Keep dependencies updated; a large share of front-end vulnerabilities come from outdated packages, not custom code.

---

13. Custom Hooks

A function starting with `use` that extracts and reuses stateful logic across components.

Example — `useDebounce`

```
function useDebounce(value, delay) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const timer = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(timer); // cancels stale pending updates
  }, [value, delay]);
  return debounced;
}
```

The cleanup clears the previous timer whenever the value changes (or on unmount), so only the latest value is ever committed — preventing redundant API calls.

Other common custom hooks: `useFetch()`, `useLocalStorage()`, `useWindowSize()`.

Debounce vs Throttle

Debounce	Throttle
Waits until the user stops triggering the event	Executes at a fixed interval while the event continues
Best for search/autocomplete inputs	Best for scroll, resize, mouse-move

---

14. Batching

React groups multiple state updates into a single re-render instead of one re-render per update.

- **Before React 18**: only updates inside native React event handlers were batched; updates inside `setTimeout`, promises, or native event listeners each triggered a separate re-render.
- **React 18**: **automatic batching** everywhere — event handlers, `setTimeout`, `Promise.then`, `async/await` — all batched into one re-render.

```
setTimeout(() => {
  setCount(c => c + 1);
  setName("Sapna");
}, 1000);
// React 18: 1 re-render. Pre-18: 2 re-renders.
```

---

15. Stale Closures

A stale closure happens when a function (inside `useEffect`, `setTimeout`, `setInterval`) captures an outdated state/prop value because it was created during an earlier render and never recreated.

Fixes: include the correct dependencies in the dependency array; use functional state updates (`setCount(prev => prev + 1)`); use `useRef` to always read the latest value inside a long-lived callback.

---

16. Lists & Keys

Keys uniquely identify list elements across renders so React can tell what was added, removed, or reordered, updating only the changed DOM nodes instead of the whole list. Always use a stable id — not the array index (see §5).

---

17. StrictMode

A development-only wrapper that surfaces potential issues: unsafe lifecycle methods, deprecated APIs, missing effect cleanup, unintended side effects. In React 18, it intentionally mounts → unmounts → mounts components again and runs effects twice in development, to help catch missing cleanup logic. **No effect in production builds.**

---

18. Client-Side Pagination

```
const [page, setPage] = useState(1);
const pageSize = 3;
const start = (page - 1) * pageSize;
const paginatedUsers = users.slice(start, start + pageSize);

<button disabled={page === 1} onClick={() => setPage(page - 1)}>Prev</button>
<button disabled={start + pageSize >= users.length} onClick={() => setPage(page + 1)}>Next</button>
```

For large datasets, prefer **server-side pagination** so the frontend never fetches more than the current page needs.

---

19. Testing *(added)*

Tool	Purpose
Jest	Test runner, assertions, mocking
React Testing Library (RTL)	Renders components and queries them the way a user would (by role/text, not implementation detail)
`@testing-library/user-event`	Simulates realistic user interactions (typing, clicking)

```
import { render, screen, fireEvent } from "@testing-library/react";

test("increments count on click", () => {
  render(<Counter />);
  fireEvent.click(screen.getByRole("button", { name: /increment/i }));
  expect(screen.getByText("Count: 1")).toBeInTheDocument();
});
```

Guiding principle (RTL philosophy): test what the user sees and does, not internal component state or implementation details — this keeps tests resilient to refactors.

---

20. Quick Interview Answers (verbatim-friendly)

Q: What is reconciliation? Reconciliation is the process React uses to compare the previous Virtual DOM with the new Virtual DOM, identify the differences, and update only the changed parts of the real DOM. Fiber is the engine that performs this efficiently.

Q: How do you optimize React performance? Minimize unnecessary re-renders and reduce React's workload: reusable components to avoid duplication, efficient shared-state management (Context or Redux Toolkit) to avoid prop drilling, server-side pagination so only needed data loads, `React.memo` / `useMemo` / `useCallback` where profiling shows a real win, code-splitting/lazy loading, debouncing/throttling high-frequency events, and stable keys in lists.

Q: How do you handle API calls in a component? Fetch inside `useEffect` (runs after render) using `async/await`, store the response in state, and track loading/error state for a good UX. Axios or `fetch` both work; cancel in-flight requests on unmount with `AbortController`.

Q: Have you used SSR? Answer honestly based on your actual experience — see §6 for a sample answer if you haven't.

---

21. Frequently Asked Questions (Common Interview Q&A) *(merged from Infosys React interview set)*

Short, sample-answer style — good for verbal answers. Several overlap with earlier sections; cross-references point back to the fuller explanation.

Q1. What is React, and what advantages does it offer over other front-end frameworks? React is a JavaScript library for building user interfaces, focused mainly on the view layer — it's not a full framework like Angular, so it's more flexible and easier to adopt gradually. Key advantages: Virtual DOM for fast updates, reusable components for maintainability, and incremental adoption without rewriting an entire codebase. (See §1, §2.)

Q2. What is JSX? JSX (JavaScript XML) is a syntax extension that lets you write HTML-like markup directly inside JavaScript, making UI code more readable than nested `createElement` calls. It compiles down to `React.createElement()` calls under the hood.

Q3. What are React components? Reusable, self-contained building blocks of the UI. They let you split an interface into independent pieces that manage their own rendering logic, making development and testing easier.

Q4. What are functional components vs class components? Functional components are plain functions that return JSX and use Hooks for state/lifecycle. Class components use ES6 classes with `this.state` and lifecycle methods (`componentDidMount`, etc.). Functional components + Hooks are the modern standard; class components are mostly seen in legacy code now.

Q5. What are React Hooks? Explain useState and useEffect. Hooks let functional components use state and lifecycle features without classes. `useState` returns `[value, setter]` for local state (§3.1). `useEffect` runs side effects (data fetching, subscriptions, manual DOM work) after render, and can clean up via a returned function (§3.2).

Q6. What is state management, and how do you manage state in React? State management means handling and updating the data that drives what the UI renders. In function components, `useState` / `useReducer` manage local state; `useContext` or a library (Redux Toolkit, Zustand) manages state shared across many components (§7).

Q7. What is the Virtual DOM, and how does it work? A lightweight in-memory representation of the real DOM. React diffs the new Virtual DOM against the previous one, then applies only the minimal set of real DOM changes — far cheaper than re-rendering everything directly (§2).

Q8. How does React handle events? Through a synthetic event system — a cross-browser wrapper around native DOM events. It normalizes behavior across browsers, uses camelCase naming (`onClick` vs `onclick`), and (in older React versions) pooled event objects for performance.

Q9. What is the difference between props and state? Props are read-only data passed from a parent to a child component. State is local data owned and updated by the component itself, and changing it triggers a re-render. Props flow down; state lives within.

Q10. What is the purpose of keys in React? Keys uniquely identify list items across renders so React can tell what was added, removed, or reordered, and update the DOM efficiently instead of re-rendering the whole list (§5, §16).

Q11. What are the lifecycle methods in React? For class components: `componentDidMount` (after first render), `componentDidUpdate` (after updates), `componentWillUnmount` (before removal), plus others like `shouldComponentUpdate`. For function components, `useEffect` (with the right dependency array) covers all three cases (§3.2).

Q12. How do you handle form/page submissions in React? Use controlled components — form values live in state via `useState`, an `onSubmit` handler prevents the default browser submission (`e.preventDefault()`), validates the data, and then sends it to the server or updates app state (§8).

Q13. What techniques maximize React performance? `React.memo` to skip re-rendering functional components when props are unchanged; `PureComponent` / `shouldComponentUpdate` for the class equivalent; lazy loading (`React.lazy` + `Suspense`) to shrink initial bundle size; and reducing unnecessary state updates (§5, §10).

Q14. What is the purpose of React Router? Provides client-side routing for single-page apps — rendering different components based on the URL without a full page reload, enabling smooth navigation between views (§9).

Q15. How do you manage large/complex state? Libraries like Redux Toolkit or Zustand, or React's built-in Context API paired with `useReducer`, to centralize state logic and avoid prop drilling across deeply nested components (§7).

Q16. What is the purpose of memoization in React? Caches the result of an expensive computation or a component's rendered output, reusing it when the same inputs (props/dependencies) recur — avoids redundant work. `useMemo` memoizes values, `useCallback` memoizes functions, `React.memo` memoizes whole components (§3.8–3.10).

Q17. Explain Higher-Order Components (HOC). A pattern (not a React API feature) where a function takes a component and returns a new component with added behavior — used to share cross-cutting logic like auth checks or data fetching across multiple components without duplicating code (§4.1).

Q18. How do you handle asynchronous operations in React? Typically inside `useEffect`, using `async/await` for clean syntax, with loading/error state tracked alongside the data — so components can fetch without blocking rendering.

Q19. Why does the `key` prop matter in lists specifically? Without stable keys, React can't reliably track which DOM node corresponds to which list item, which can cause it to lose or reuse component state incorrectly when the list changes, and hurts reconciliation performance (§5, §16).

Q20. How do you handle error boundaries in React? Create a class component defining `static getDerivedStateFromError()` and/or `componentDidCatch()`, and render a fallback UI in place of the crashed subtree instead of letting the whole app crash (§4.3).

Q21. What is `useRef` and how is it used? Persists a mutable value across renders without causing a re-render when it changes. Common uses: referencing a DOM node directly (focus, scroll, measure), or storing a timer ID or previous value (§3.4).

Q22. How do you make React components reusable? Keep components focused on one responsibility, favor composition (children/render props) over duplicating logic, extract shared stateful logic into custom hooks, and design props to be flexible rather than overly specific to one use case (§4, §13).

Summary Table — What to Reach For

Problem	Solution
Prop drilling	Context
Complex/related state	useReducer
Expensive computation	useMemo
Function identity for a memoized child	useCallback
Skipping a child’s re-render	React.memo
DOM access / mutable value without re-render	useRef
Imperative API for a child	forwardRef + useImperativeHandle
Escaping overflow/z-index (modals)	Portals
Crash containment	Error Boundary
Large bundle / route-based loading	React.lazy + Suspense
Non-urgent UI update during fast typing	useTransition / useDeferredValue
Global state, small/medium app	Context + useReducer, or Zustand
Global state, large app with async flows	Redux Toolkit